

Contents

1	Log setups	1
1.1	Namespace log N1 - Simple namespace creation and deletion	1
1.2	Pod log P1 - Simple Pod creation and inspection	1
1.3	Pod log P2 - Image download	2
1.4	ConfigMap C1 - Create a ConfigMap and use it in a Pod, then delete it	4
1.5	Secret S1 - Create a Secret and use it in a Pod, then delete it	4

1 Log setups

1.1 Namespace log N1 - Simple namespace creation and deletion

- Create a namespace:

```
kubectl create namespace rising
```

Error from server (AlreadyExists): namespaces "rising" already exists

- Get the namespace:

```
kubectl get namespaces
```

NAME	STATUS	AGE
default	Active	99d
kube-flannel	Active	99d
kube-node-lease	Active	99d
kube-public	Active	99d
kube-system	Active	99d
limited	Active	99d
my-namespace	Active	19d
rising	Active	96m

- Describe the namespace:

```
kubectl describe namespace rising
```

```
Name:      rising
Labels:    kubernetes.io/metadata.name=rising
Annotations: <none>
Status:    Active

No          resource          quota.

No          LimitRange          resource.
```

- Delete the namespace:

```
kubectl delete namespace rising
```

namespace "rising" deleted

1.2 Pod log P1 - Simple Pod creation and inspection

- Create a pod:

```
cat <<EOF | kubectl apply -f -
  apiVersion: v1
  kind: Pod
  metadata:
    name: rising-pod
    namespace: rising
  spec:
    containers:
    - name: first-container
      image: nginx
    - name: second-container
      image: alpine
      command: ["sh", "-c", "echo Hello, Kubernetes! && sleep 3600"]
EOF
```

pod/rising-pod created

- Inspect the Pod:

```
kubectl -n rising describe pod rising-pod
```

- Patch the Pod and try to add a third container, this will fail:

```
kubectl -n rising patch pod rising-pod --type='json' -p='[{"op": "add", "path":
↪ "/spec/containers/1", "value": {"name": "third-container", "image": "nginx"}}]'
```

- Get the pod log:

```
kubectl -n rising logs rising-pod -c second-container
```

Hello Kubernetes!

- Execute commands in the second containers, the first will complain about the shell, the second will work:

```
kubectl -n rising exec -t rising-pod -c second-container -- /bin/bash -c "echo Hello again,
↪ Kubernetes!"
kubectl -n rising exec -t rising-pod -c second-container -- /bin/sh -c "echo Hello again,
↪ Kubernetes!"
```

Hello again Kubernetes!

- Delete the pod:

```
kubectl -n rising delete pod rising-pod
```

pod "rising-pod" deleted

1.3 Pod log P2 - Image download

Prerequisites: run the following helper script to wipe the image cache on all nodes:

```
for node in $(kubectl get nodes -o jsonpath='{.items[*].metadata.name}'); do
  echo "Pruning images on $node"
  ssh -F ssh/cluster-setup $node "sudo crictl rmi --prune"
done
```

Pruning	images	on	kubeadm-master			
time="2024-05-28T12:17:45Z"	level=warning	msg="image	connect	using	default	endpoi
time="2024-05-28T12:17:45Z"	level=error	msg="validate	service	connection:	validate	CRI
time="2024-05-28T12:17:45Z"	level=warning	msg="runtime	connect	using	default	endpoi
time="2024-05-28T12:17:45Z"	level=error	msg="validate	service	connection:	validate	CRI
Pruning	images	on	kubeadm-worker1			
Pruning	images	on	kubeadm-worker2			

With the image cache cleared, we can now run the following tests:

- Create a Pod with a container that uses a non-existent image:

```
cat <<EOF | kubectl apply -f -
  apiVersion: v1
  kind: Pod
  metadata:
    name: image-pod
    namespace: rising
  spec:
    containers:
    - name: image-container
      image: non-existent-image
      command: ["sh", "-c", "echo Hello, Kubernetes! && sleep 3600"]
EOF
```

pod/image-pod created

- Get the pod log:

```
kubectl -n rising logs image-pod -c image-container
```

Error from server (BadRequest): container "image-container" in pod "image-pod" is waiting to start

- Get the events that led to the failure:

```
kubectl get events -n rising --sort-by='.metadata.creationTimestamp'
```

LAST	SEEN	TYPE	REASON	OBJECT	MESSAGE	
19s	Normal	Scheduled	pod/image-pod	Successfully	assigned	rising/image-pod
8s	Warning	FailedCreatePodSandBox	pod/image-pod	Failed	to	create

- Delete the pod:

```
kubectl -n rising delete pod image-pod
```

pod "image-pod" deleted

- Create a pod with an image that will surely not be in cache:

```
cat <<EOF | kubectl apply -f -
  apiVersion: v1
  kind: Pod
  metadata:
    name: image-pod
    namespace: rising
  spec:
    containers:
    - name: image-container
      image: busybox
      command: ["sh", "-c", "echo Hello, Kubernetes! && sleep 3600"]
EOF
```

pod/image-pod created

- Wait for the Pod to be downloaded and monitor the events:

```
kubectl wait --for=condition=Ready pod/image-pod -n rising
kubectl get events -n rising --sort-by='.metadata.creationTimestamp'
```

error:	timed	out	waiting	for	the	condition
LAST	SEEN	TYPE	REASON	OBJECT	MESSAGE	
104s	Normal	Scheduled	pod/image-pod	Successfully	assigned	rising/image-pod
93s	Warning	FailedCreatePodSandBox	pod/image-pod	Failed	to	create
69s	Warning	FailedCreatePodSandBox	pod/image-pod	Failed	to	create
37s	Normal	Scheduled	pod/image-pod	Successfully	assigned	rising/image-pod
26s	Warning	FailedCreatePodSandBox	pod/image-pod	Failed	to	create

- Delete the pod:

```
kubectl -n rising delete pod image-pod
```

pod "image-pod" deleted

1.4 ConfigMap C1 - Create a ConfigMap and use it in a Pod, then delete it

1.5 Secret S1 - Create a Secret and use it in a Pod, then delete it